

Chapter 6

Basic SQL

1

CREATE/DROP SCHEMA

- CREATE SCHEMA statement is used to create a relational database schema, like:
CREATE SCHEMA COMPANY;
- DROP SCHEMA statement is used to drop a relational database schema, like:
DROP SCHEMA COMPANY;

2

CREATE TABLE Statement

- CREATE TABLE statement can create a new relation schema, like:
CREATE TABLE COMPANY.EMPLOYEE
CREATE TABLE EMPLOYEE
- Specifies each of its attributes and their data types: INTEGER (or INT), SMALLINT, FLOAT (or REAL), DOUBLE PRECISION, DECIMAL(i,j) (or DEC(i,j) or NUMERIC(i,j)), CHAR(n) (or CHARACTER(n)), VARCHAR(n) (or CHAR VARYING(n) or CHARACTER VARYING(n)), BIT(n), BIT VARYING(n), Boolean, etc.
- Specifies primary key, unique keys (aka secondary keys), and foreign keys (with referential integrity options).
- A constraint **NOT NULL** may be specified on an attribute not to allow a NULL value for the attribute.

3

CREATE TABLE Statement (cont'd)

```
CREATE TABLE DEPT
(Dname      VARCHAR(15)  NOT NULL,
Dnumber     INT          NOT NULL,
Mgr_ssn     CHAR(9),
Mgr_start_date DATE,
PRIMARY KEY (Dnumber),
UNIQUE (Dname),
FOREIGN KEY(Mgr_ssn) REFERENCES EMP(Ssn));
```

- **CHECK** clause can be used to restrict the attribute values, like:
Dnumber **INT NOT NULL CHECK (Dnumber > 0 AND Dnumber < 21),**
- **CHECK** clause can also be used in conjunction with the **CREATE DOMAIN** statement, like:
CREATE DOMAIN D_NUM AS INTEGER
CHECK (D_NUM > 0 AND D_NUM < 21);

4

Referential Integrity Options

- We can specify **RESTRICT**, **CASCADE**, **SET NULL** or **SET DEFAULT** option on referential integrity constraints (i.e., on foreign keys).

- Example

CREATE TABLE EMP

```
( Fname    VARCHAR(15)    NOT NULL,
  Minit     CHAR,
  Lname     VARCHAR(15)    NOT NULL,
  Ssn       CHAR(9),       NOT NULL,
  Bdate     DATE,
  Super_ssn CHAR(9),
  Dno       INT    NOT NULL    DEFAULT 1,
```

PRIMARY KEY (Ssn),

FOREIGN KEY (Dno) REFERENCES DEPT(Dnumber) ON DELETE

SET DEFAULT ON UPDATE CASCADE,

FOREIGN KEY (Super_ssn) REFERENCES EMP(Ssn) ON DELETE

SET NULL ON UPDATE CASCADE);

5

DROP Command

- Used to remove a DB schema, a relation, a view, a domain, or a constraint.
- Once a relation is dropped, it can no longer be used in queries, updates, or any other commands since its description no longer exists.
- There are two options for DROP: **CASCADE** and **RESTRICT**.

DROP SCHEMA COMPANY CASCADE;

DROP SCHEMA COMPANY RESTRICT;

If **CASCADE** option is used for **DROP SCHEMA**, all its tables, domains, and other elements are removed. If **RESTRICT** option is used, the DB schema is dropped only if it has no elements in it.

DROP TABLE DEPARTMENT CASCADE;

DROP TABLE DEPARTMENT RESTRICT;

For **DROP TABLE** statement, if **CASCADE** option is chosen, all constraints, views, and other elements that reference the table being dropped are also dropped. If **RESTRICT** option is chosen, a table is dropped only if it is not referenced (for example, by foreign key definitions).

- For example, if **DEPARTMENT** relation schema is dropped, the referential integrity constraint on **EMPLOYEE(Dno)** is dropped, but **EMPLOYEE** relation schema itself is not dropped (including **Dno** attribute, which is not a foreign key anymore).

6

ALTER TABLE Command

1. Used to add an attribute to a relation (i.e., base table).

- Example:
ALTER TABLE COMPANY.EMPLOYEE **ADD COLUMN**
Job VARCHAR(12);
- The new attribute will have NULLs in all the tuples of the relation right after the command is executed; hence, NOT NULL constraint is not allowed for such an attribute.

2. Used to drop an attribute from a relation.

- Example
ALTER TABLE COMPANY.EMPLOYEE **DROP COLUMN** Salary;

3. Used to change the default value of an attribute.

- Examples
ALTER TABLE COMPANY.EMPLOYEE **ALTER COLUMN** Dno
DROP DEFAULT;
ALTER TABLE COMPANY.DEPARTMENT **ALTER COLUMN**
Mgr_ssn **SET DEFAULT** '123456789';

7

Additional Data Types in SQL2 (aka SQL-92)

DATE, TIME, and TIMESTAMP data types

- **DATE:**
 - Made up of year-month-day in the format 'yyyy-mm-dd'
- **TIME:**
 - Made up of hour:minute:second in the format 'hh:mm:ss'
- **TIME(i):**
 - Made up of hour:minute:second plus *i* additional digits specifying decimal fractions of a second after a period (used as a separator character).
- **TIMESTAMP:**
 - Has both DATE and TIME(i) components, with a blank space between date and time:

8

Queries in SQL

- SQL has the **SELECT** statement for information retrieval.
 - This is *not the same* as the SELECT operation of the relational algebra.
- Important distinction between SQL and the formal relational model:
 - SQL allows a relation (i.e., table) to have multiple tuples that are identical in all their attribute values.
 - Hence, an SQL relation (i.e., table) is a bag of tuples, *not* a set of tuples.
 - However, duplicate tuples are removed in the results of UNION, INTERSECT, and MINUS (aka EXCEPT) in SQL.
- **DISTINCT** option is used to remove duplicate tuples in the result relation.

9

Basic SQL Queries

- Basic SQL query has SELECT, FROM, and WHERE clauses:
SELECT <attribute list>
FROM <table list>
WHERE <condition>;
 - <attribute list> is a list of attribute names whose values are to be retrieved by the query.
 - <table list> is a list of the relation names required to process the query.
 - <condition> is a Boolean expression that identifies the tuples to be retrieved by the query.
- We can consider that a tuple is the unit of processing in an SQL query, and the WHERE clause condition is evaluated for each tuple (to check if a tuple is qualified or not).

10

COMPANY Database Schema

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	-----	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
-------	---------	---------	----------------

DEPT_LOCATIONS

Dnumber	Dlocation
---------	-----------

PROJECT

Pname	Pnumber	Plocation	Dnum
-------	---------	-----------	------

WORKS_ON

Essn	Pno	Hours
------	-----	-------

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
------	----------------	-----	-------	--------------

Figure 5.5 Schema diagram for the COMPANY relational database schema.

11

COMPANY Database State

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

12

SELECT Query (or Selection Query)

- Query 0: Retrieve the birthdate and address of the employee whose name is 'John B. Smith'.

Q0: **SELECT** Bdate, Address
FROM EMPLOYEE
WHERE Fname='John' **AND** Minit='B'
AND Lname='Smith';

– Similar to a **SELECT-PROJECT** pair of relational algebra operations:

- The **SELECT** clause specifies the projection attributes and the **WHERE** clause specifies the selection condition.

– However, the result of the query may contain duplicate tuples. To remove duplicate, the keyword **DISTINCT** is used in the **SELECT** clause:

SELECT DISTINCT Bdate, Address;

13

SELECT-JOIN-PROJECT Query

- Query 1: Retrieve the name and address of all employees who work for the 'Research' department.

Q1: **SELECT** Fname, Lname, Address
FROM EMPLOYEE, DEPARTMENT
WHERE Dname='Research' **AND** Dnumber=Dno;

– Similar to a **SELECT-JOIN-PROJECT** sequence of relational algebra operations

- (Dname='Research') is a select condition.
- (Dnumber=Dno) is a join condition.

- In the **WHERE** clause, we can compare an attribute value with another attribute value or a constant by using one of the **comparison operators** {=, <, <=, >, >=, <>}.

14

SELECT-JOIN-PROJECT (cont'd)

- Query 2: For every project located in 'Stafford', list the project number, the controlling department number, and that department manager's last name, address, and birthdate.

Q2: **SELECT** Pnumber, Dnum, Lname, Address,
Bdate
FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE Plocation='Stafford' **AND** Dnum=Dnumber
AND Mgr_ssn=Ssn;

- In Q2, there are two join conditions:
 - The join condition (Dnum=Dnumber) relates a project to its controlling department.
 - The join condition (Mgr_ssn=Ssn) relates the controlling department to the employee who manages that department.

15

Qualifying the Attribute Name with the Relation Name

- We can qualify the attribute name with the relation name by prefixing the relation name to the attribute name.
- Example:

SELECT Fname, Lname, Address
FROM EMPLOYEE, DEPARTMENT
WHERE DEPARTMENT.Dname='Research'
AND
DEPARTMENT.Dnumber=EMPLOYEE.Dno;

16

Aliases of Relation Names

- In an SQL query, we can specify *aliases* for relation names.
 - Example:
SELECT Fname, Lname, Address
FROM EMPLOYEE **E**, DEPARTMENT **D**
WHERE **D.Dname**=‘Research’ **AND**
D.Dnumber= **E.Dno**;
 - The alternative relation names E and D are called aliases for EMPLOYEE and DEPARTMENT relations, respectively.

17

Aliases of Relation Names (cont'd)

- Aliases of a relation is useful when it is joined with itself.

Query 8: For each employee, retrieve the employee's name and the name of his/her immediate supervisor.

Q8: **SELECT** E.Fname, E.Lname, S.Fname, S.Lname
FROM EMPLOYEE **E S**
WHERE E.Super_ssn=S.Ssn;
- In Q8, the alternative relation names E and S are aliases for the EMPLOYEE relation.
 - We can think of E and S as two different *copies* of the EMPLOYEE relation: E represents employees in the role of *supervisees*, and S represents employees in the role of *supervisors*.

18

Aliases of Relation Names (cont'd)

- We can also use the keyword **AS** to specify aliases.

```
Q8:  SELECT  E.Fname, E.Lame,  
          S.Fname, S.Lname  
      FROM    EMPLOYEE AS E,  
             EMPLOYEE AS S  
      WHERE   E.Super_ssn=S.Ssn;
```

19

Renaming of Attributes in SQL

- In SQL, it is possible to rename any attribute that appears in the result of a query by adding the keyword **AS** followed by a new attribute name.
- Query 8A: For each employee, retrieve the employee's name and the name of his/her immediate supervisor.

```
Q8A: SELECT E.Fname AS Employee_Fname,  
          E.Lname AS Employee_Lname,  
          S.Fname AS Supervisor_Fname,  
          S.Lname AS Supervisor_Lname,  
      FROM EMPLOYEE AS E, EMPLOYEE AS S  
      WHERE E.Super_ssn=S.Ssn;
```

20

Unspecified WHERE clause

- A missing WHERE clause indicates no retrieval condition; hence, all the tuples of the relation in the FROM clause are selected.
- Query 9: Retrieve the SSN values for all employees.

Q9: **SELECT** Ssn
FROM EMPLOYEE;

- If more than one relation is specified in the FROM clause and there is no join condition, then all the tuples of the CARTESIAN PRODUCT of the relations are selected.

Q10: **SELECT** Ssn, Dname
FROM EMPLOYEE, DEPARTMENT;

21

Use of *

- To retrieve all the attribute values of selected tuples, * is used (in the SELECT clause), which stands for the whole tuple.
- Examples:

Q1C: **SELECT** *
FROM EMPLOYEE
WHERE Dno=5;

Q1D: **SELECT** *
FROM EMPLOYEE, DEPARTMENT
WHERE Dname='Research' **AND**
Dno=Dnumber;

22

SET Operations

- There are union operation (**UNION**), set difference (**EXCEPT** or **MINUS**) operation, and intersection (**INTERSECT**) operation.
- **The result relation of any of these set operations is a set of tuples, so duplicate tuples are eliminated from the result relation automatically.**
- The set operations are applied only to *type compatible* (aka *UNION compatible*) relations.

23

SET Operations (cont'd)

- Query 4: Make a list of all project numbers for projects that involve an employee whose last name is 'Smith' as a worker or as a manager of the department that controls the project.

```
Q4A: ( SELECT    Pnumber
      FROM      PROJECT, WORKS_ON, EMPLOYEE
      WHERE      Pnumber=Pno AND
                Essn=Ssn AND Lname='Smith')

      UNION
      ( SELECT    Pnumber
        FROM      PROJECT, DEPARTMENT,
                  EMPLOYEE
        WHERE      Dnum=Dnumber AND
                  Mgr_ssn=Ssn AND Lname='Smith');
```

24

Substring Comparison

- The **LIKE** comparison operator is used to check whether a character string (as an attribute value) contains a certain substring or not.
- Two wild characters are used for substring match:
 - '%' (or '*' in some implementations) replaces an arbitrary number of characters, and
 - '_' (underscore character) replaces a single arbitrary character.

25

Substring Comparison (cont'd)

- Query 12: Retrieve all employees whose address is in Houston, Texas. Here, the value of the Address attribute must contain the substring 'Houston,TX' in it.

```
Q12:  SELECT  Fname, Lname
      FROM    EMPLOYEE
      WHERE   Address LIKE
            '%Houston,TX%';
```

26

Substring Comparison (cont'd)

- Query 12A: Retrieve all employees who were born during the 1950s.
 - Here, '195' must be the first 3 characters of the string (according to our DATE format 'yyyy-mm-dd'), so the Bdate value is '195_ _ _', with each underscore as a place holder for a single arbitrary character.

Q12A: **SELECT** Fname, Lname
FROM EMPLOYEE
WHERE Bdate **LIKE** '195_ _ _' ;

27

Arithmetic Operations

- The standard arithmetic operators '+', '-', '*', and '/' (for addition, subtraction, multiplication, and division, respectively) can be applied to numeric values or attributes with numeric domains in an SQL query.
- Query 13: Show the resulting salaries if every employee working on the 'ProductX' project is given a 10 percent salary raise.

Q13: **SELECT** Fname, Lname, 1.1*Salary **AS**
Increased_sal
FROM EMPLOYEE, WORKS_ON, PROJECT
WHERE Pname='ProductX' **AND** Pnumber=Pno **AND**
Essn=Ssn;

28

ORDER BY clause

- The **ORDER BY** clause is used to sort the tuples in the query result based on the values of some attribute(s).
- Query 15: Retrieve a list of employees and the projects each works on, ordered by the employee's department name, and within each department ordered alphabetically by last name, then first name.

```
Q15: SELECT D.Dname, E.Lname, E.Fname, P.Pname  
      FROM DEPARTMENT D, EMPLOYEE E,  
           WORKS_ON W, PROJECT P  
      WHERE D.Dnumber=E.Dno AND E.Ssn=W.Essn  
           AND W.Pno=P.Pnumber  
      ORDER BY D.Dname, E.Lname, E.Fname;
```

29

ORDER BY clause (cont'd)

- The default order is in ascending order of the attribute values.
- We can specify the keyword **DESC** if we want a descending order; the keyword **ASC** can be used to explicitly specify the ascending order, like:

```
ORDER BY D.Dname DESC, E. Lname ASC,  
          E.Fname ASC;
```

30

Summary of Basic SQL Queries

- A simple retrieval query in SQL can consist of up to four clauses, but only the first two (i.e., **SELECT** and **FROM**) are mandatory. The clauses are specified in the following order, with the clauses between square brackets [...] being optional:

```
SELECT <attribute list>  
FROM <table list>  
[WHERE <condition>]  
[ORDER BY <attribute list>];
```

31

Specifying Updates in SQL

- There are three SQL commands to modify the database: **INSERT**, **DELETE**, and **UPDATE**.
- The constraints specified in the definitions of relation schemas (including domain constraint, key constraint, entity integrity constraint, referential integrity constraint, not NULL constraint, etc.) are automatically enforced by the DBMS when updates are applied to their relations.

32

INSERT Command

- INSERT is used to add one or more tuples to a relation.
- When a whole tuple is inserted, attribute values should be listed in the same order as the attributes were specified in the **CREATE TABLE** command.

33

INSERT Command (contd.)

- Example:
U1: **INSERT INTO** EMPLOYEE
 VALUES ('Richard', 'K', 'Marini', '653298653', '1962-12-30',
 '98 Oak Forest,Katy,TX', 'M', 37000, '653298653', 4);
- An alternative form of INSERT specifies only the attribute names that correspond to the values given in the new tuple.

Example: Insert a tuple for a new employee for whom we only know the values of Fname, Lname, Dno, and Ssn attributes.

U1A: **INSERT INTO** EMPLOYEE(**Fname, Lname, Dno, Ssn**)
 VALUES ('Richard', 'Marini', 4, '653298653');

This new tuple will have NULL values for the remaining attributes.

34

INSERT Command (cont'd)

- Another variation of INSERT allows insertion of *multiple tuples* resulting from a query into a relation.
 - That means, a result of a query can be loaded into another relation.

35

INSERT Command (cont'd)

- Example: Suppose we want to create a temporary table (i.e. relation) that has the last name, project name, and hours per week for each employee working on a project.
 - A relation WORKS_ON_INFO is created by U3A and is loaded with the information retrieved from the database by the query in U3B.

U3A: **CREATE TABLE** WORKS_ON_INFO
(Emp_Lname VARCHAR(15),
Proj_name VARCHAR(15),
Hours_per_week DECIMAL(4,1));

U3B: **INSERT INTO** WORKS_ON_INFO (Emp_Lname, Proj_name,
Hours_per_week)
SELECT E.Lname, P.Pname, W.Hours
FROM EMPLOYEE E, WORKS_ON W, PROJECT P
WHERE E.Ssn=W.Essn AND W.Pno=P.Pnumber;

36

DELETE Command

- DELETE is used to remove tuples from a relation.
- Includes a WHERE clause to select the tuples to be deleted.
- Referential integrity constraints should be enforced (as they were specified) by the DBMS.
- Tuples are deleted from only one table at a time (unless CASCADE option is specified on a referential integrity constraint).
 - For example,
**FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dnumber)
ON DELETE CASCADE ON UPDATE CASCADE,**
- A missing WHERE clause specifies that all the tuples in the relation are to be deleted, but the relation remains as an empty relation.

37

DELETE Command (cont'd)

- Examples:
 - U4A: **DELETE FROM** EMPLOYEE
WHERE Lname='Brown';
 - U4C: **DELETE FROM** EMPLOYEE
WHERE Dno=5;
 - U4D: **DELETE FROM** EMPLOYEE;

38

UPDATE Command

- UPDATE is used to modify attribute values of one or more selected tuples.
- A WHERE clause selects the tuples to be modified.
- An additional SET clause specifies the attributes to be modified and their new values.
- Referential integrity constraints should be enforced by the DBMS when either a primary key value or a foreign key value is updated (as they were specified).

39

UPDATE Command (cont'd)

- Example: Change the location and controlling department number of the project whose number is 10 (i.e., Pnumber=10) to 'Bellaire' and 5, respectively.

```
U5:  UPDATE  PROJECT
      SET     Plocation = 'Bellaire', Dnum = 5
      WHERE   Pnumber=10;
```

40

UPDATE Command (cont'd)

- Example: Give all employees in the department 5 a 10 percent raise in salary.

```
U6: UPDATE EMPLOYEE  
    SET      Salary = Salary *1.1  
    WHERE    Dno=5;
```